

Django Social Media Platform

Python Development Internship

Project

Submitted by

Antony John Robees A

Computer Science Graduate

Python Development Intern

Submission Date:

09 August 2026

Table of Content

Django Social Media Platform.....	1
Python Development Internship.....	1
Project.....	1
Table of Content.....	2
1. Project Overview.....	4
2. Project Objectives.....	4
3. Technology Stack.....	5
4. System Architecture.....	5
Architecture Components.....	6
Model.....	6
View.....	6
Template.....	6
Forms.....	7
5. Project Structure.....	7
6. Application Modules.....	8
6.1 Users Application.....	8
6.2 Posts Application.....	9
6.3 Friends Application.....	9
7. Database Design.....	9
7.1 User and Profile.....	10
7.2 Post.....	11
7.3 Comment.....	11
7.4 Like.....	11
7.5 Entity Relationships.....	12
8. Authentication and Authorization.....	13
Registration.....	13
Login.....	14
Logout.....	14
Protected Views.....	14
9. User and Profile Workflow.....	15
10. Post Workflow.....	15
Creating a Post.....	16
Editing a Post.....	16
Deleting a Post.....	16
11. Comment and Like Workflow.....	16
Comments.....	16
Likes.....	17
12. Forms and Validation.....	17
UserUpdateForm.....	18

ProfileUpdateForm.....	18
Date of Birth Validation.....	18
Password Update.....	18
13. Django Signals.....	19
14. URL Routing.....	20
15. Templates and Presentation Layer.....	20
16. Media and Static Files.....	21
Profile Images.....	21
Post Images.....	21
17. Messages and User Feedback.....	22
18. Security and Access Control.....	22
Authentication Protection.....	22
CSRF Protection.....	22
Ownership Validation.....	23
Form Validation.....	23
19. Current Implementation Status.....	23
20. Known Limitations.....	24
Home Feed.....	24
Friends.....	24
Notifications.....	24
REST API.....	25
Automated Tests.....	25
Development Environment.....	25
21. Future Enhancements.....	25
Friends and Relationships.....	25
Home Feed.....	25
Notifications.....	25
REST API.....	26
Automated Testing.....	26
Frontend Improvements.....	26
22. Development Decisions and Lessons Learned.....	27
Separate Profile Model.....	27
Automatic Profile Creation.....	27
Model-Based Forms.....	27
Authentication-Based Access.....	27
Ownership-Based Operations.....	28
Separation of Applications.....	28
Media Handling.....	28
23. Conclusion.....	28

1. Project Overview

The Django Social Media Platform is a web application developed using Django that provides core social media functionality for authenticated users.

The current version allows users to:

- Register and authenticate
- Maintain a personal profile
- Upload a profile picture
- Create posts
- Add optional images to posts
- Edit and delete their own posts
- Like and unlike posts
- Add comments
- Delete their own comments

The application uses Django's built-in authentication system together with custom models for profiles, posts, comments, and likes.

The project follows Django's Model-View-Template (MVT) architecture.

2. Project Objectives

The primary objectives of the project are:

1. To develop a functional social media web application using Django.
2. To implement user authentication using Django's built-in authentication framework.
3. To provide users with customizable profiles.

4. To allow authenticated users to create and manage posts.
 5. To provide basic social interaction through likes and comments.
 6. To implement ownership-based access control for user-generated content.
 7. To support image uploads for profiles and posts.
 8. To apply form validation before storing user-provided data.
 9. To organize the application into separate Django applications based on functionality.
 10. To provide a foundation that can be extended with additional social networking features.
-

3. Technology Stack

Technology	Purpose
Python 3.12	Programming language
Django 6.0.7	Web framework
SQLite	Development database
HTML	Page structure
CSS	Page styling
Django Templates	Presentation layer
Django ORM	Database interaction
Django Authentication	User authentication and session management

The project's Python dependencies are maintained in [requirements.txt](#).

4. System Architecture

The project follows Django's Model-View-Template architecture.

The general request flow is:

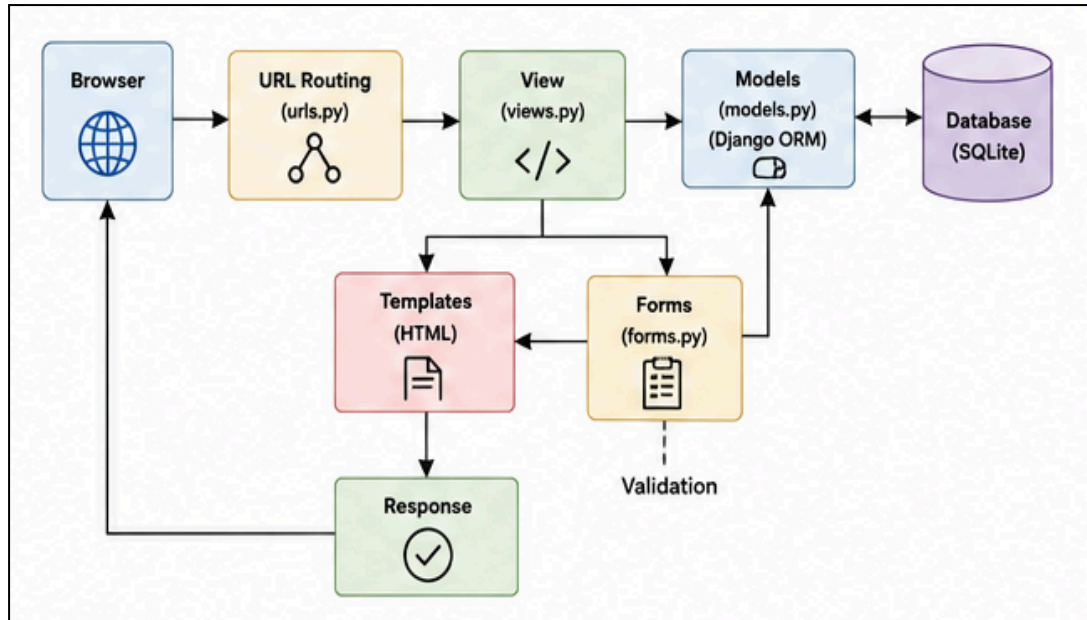


DIAGRAM 1 - System Architecture Diagram

Architecture Components

Model

- Models represent the application's data structure and provide the interface used by Django ORM to interact with the database.
- The project uses models for:
 - Profile
 - Post
 - Comment
 - Like
- The built-in Django **User** model is used for account and authentication data.

View

- Views contain the application logic responsible for processing requests, validating submitted information, interacting with models, and returning responses.

Template

- Django templates provide the HTML presentation layer.
- Templates receive information from views through the context and display it to the user.

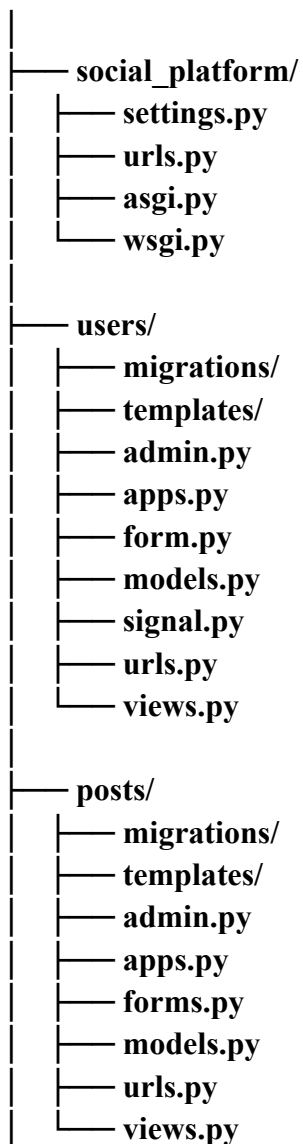
Forms

- Django forms are used to collect and validate user-provided information before it is saved.
-

5. Project Structure

The major project structure is:

Social-Media-Platform/



```
|
|— friends/
|— templates/
|— static/
|— media/
|— db.sqlite3
|— manage.py
|— requirements.txt
|— .gitignore
```

The project is divided into separate applications so that different responsibilities remain organized.

6. Application Modules

6.1 Users Application

The **users** application manages account-related functionality.

Its responsibilities include:

- User registration
- Login
- Logout
- Profile display
- Profile editing
- User information editing
- Password management
- Profile creation
- Authentication-protected user pages

The application uses Django's built-in **User** model rather than creating a separate custom authentication model.

A separate **Profile** model is associated with each User through a one-to-one relationship.

6.2 Posts Application

The **posts** application manages user-generated content and interactions.

Its responsibilities include:

- Creating posts
- Displaying posts
- Editing posts
- Deleting posts
- Post image uploads
- Comments
- Comment deletion
- Likes
- Unlikes
- Like counts

Post ownership is checked before allowing users to modify or delete their posts.

6.3 Friends Application

- The **friends** application is retained as a planned component for future development.
 - The friends system is not implemented in v1.0.
 - The intended purpose is to provide relationship functionality between users and eventually support a friend-based Home feed.
 - The current Home page therefore remains a placeholder until the friends functionality is implemented.
-

7. Database Design

- The project uses SQLite as its development database.
 - Django ORM is used to represent the application models and perform database operations.
-

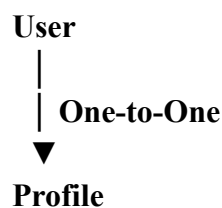
7.1 User and Profile

- The application uses Django's built-in **User** model for authentication and account information.
- The custom **Profile** model stores additional information that is not part of the default User model.

The Profile contains:

Field	Purpose
user	One-to-one relationship with User
profile_picture	User profile image
bio	Short user biography
gender	Gender selection
date_of_birth	User date of birth
phone_number	Contact number
location	User location
website	User website
is_private	Private account setting

The Profile is associated with the User using a **OneToOneField**.



7.2 Post

The Post model represents content created by authenticated users.

A post can contain:

- A title
- Post content
- An optional image
- Creation date
- Edited status
- Edited date
- The user who created it

A post belongs to its creating user.

7.3 Comment

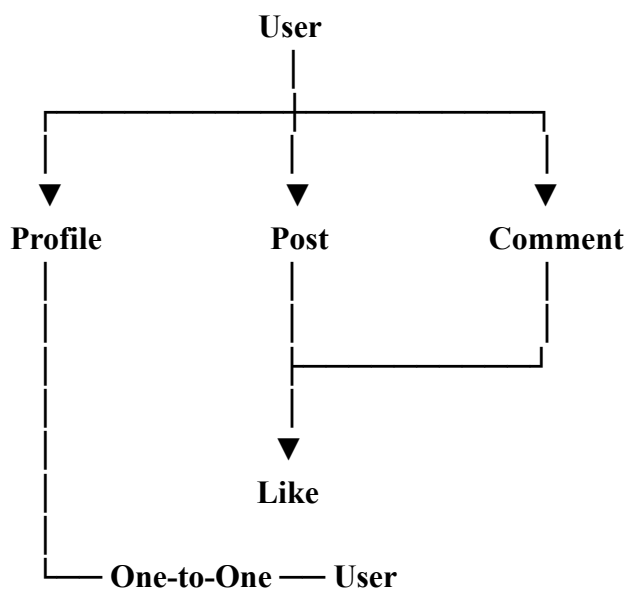
- The Comment model represents a user's response to a post.
 - A comment contains:
 - The user who created it
 - The associated post
 - Comment text
 - Creation date
 - A post can have multiple comments.
 - Users are allowed to delete comments that they own.
-

7.4 Like

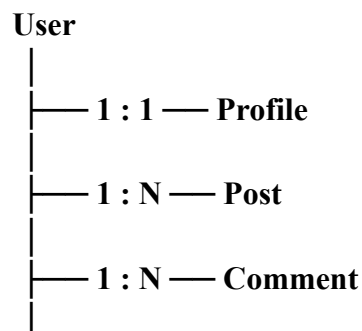
- The Like model represents a user's interaction with a post.
 - A like connects:
 - The user who liked the post
 - The post being liked
 - The application uses this relationship to determine:
 - Total likes on a post
 - Whether the current user has liked a post
 - Whether the user's like should be added or removed
-

7.5 Entity Relationships

The main relationships can be represented as:



More specifically:



└─ 1 : N ─ Like

Post

└─ 1 : N ─ Comment

└─ 1 : N ─ Like

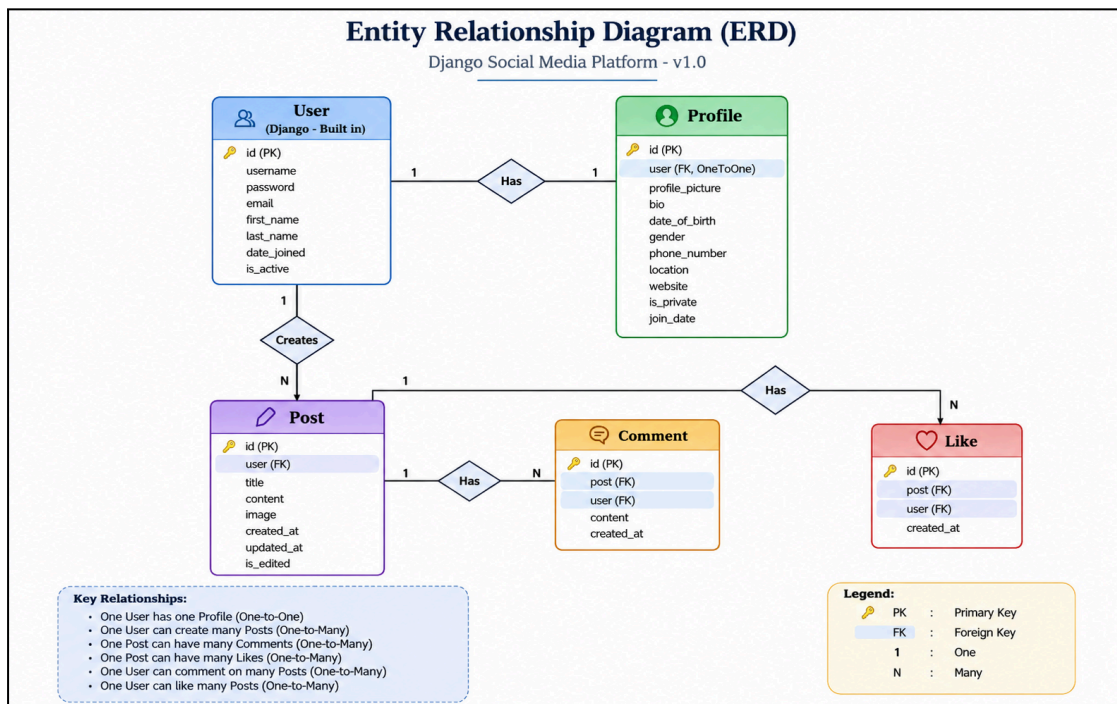


DIAGRAM 2 - Entity Relationship Diagram

8. Authentication and Authorization

- The project uses Django's built-in authentication system.
- The authentication process is handled using Django authentication forms and functions.

Registration

The **RegisterForm** extends Django's **UserCreationForm**.

The registration process collects:

- Username
- Email
- Password
- Password confirmation

The form validates the supplied information before creating the User.

Login

- The **LoginForm** is based on Django's **AuthenticationForm**.
- After form validation, the username and password are passed to Django's authentication mechanism.
- If authentication succeeds, Django creates the authenticated session and redirects the user to the Home page.

Logout

- The logout view terminates the user's authenticated session and redirects the user to the Login page.

Protected Views

Authentication-protected views use Django's **login_required** decorator.

Protected functionality includes:

- Profile
- Edit Profile
- Home
- Post-related operations
- Logout

Unauthenticated users are redirected to the Login page when accessing protected views.

9. User and Profile Workflow

The user registration and Profile creation workflow is:

User opens Registration



Registration Form



Form Validation



Django User Created



post_save Signal



Profile Created



User Redirected to Login

- When a new User is created, the **post_save** signal checks whether the object was newly created.
- If the User is newly created, a corresponding Profile is automatically created.
- This removes the need for the registration view to manually create the Profile.

10. Post Workflow

The post creation workflow is:

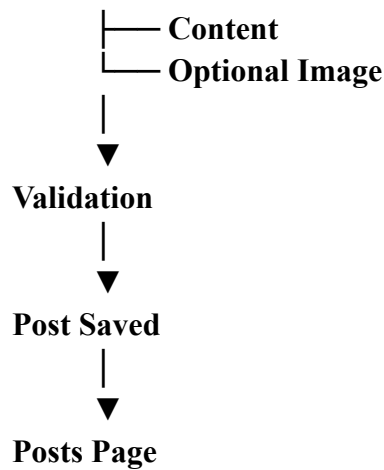
User selects Create Post



Post Form



Title



Creating a Post

Authenticated users can create a post by providing the required content and optionally adding a title and image.

Editing a Post

- The user can edit only their own post.
- The existing post information is displayed in the editing interface. The user can modify the title, content, or image.
- The application records the edited state and edited time.

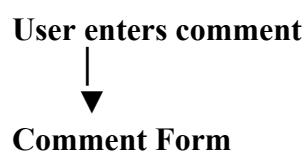
Deleting a Post

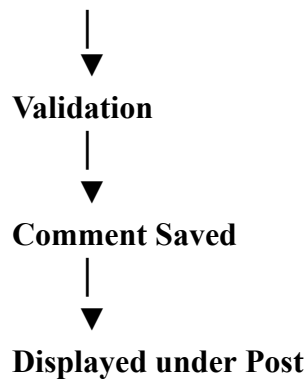
- The application checks post ownership before deletion.
- The deletion page displays the post and requests confirmation through the browser's confirmation dialog before submitting the deletion request.

11. Comment and Like Workflow

Comments

The comment workflow is:



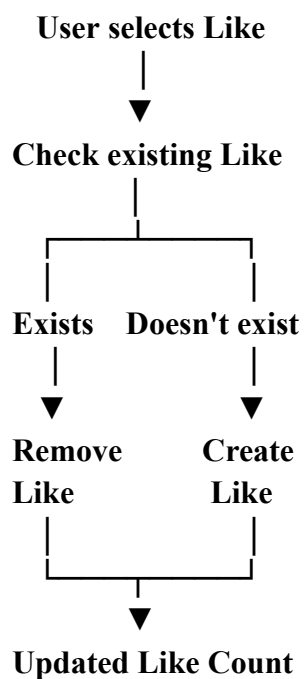


The application displays comments through the relationship between the Post and Comment models.

A user can delete a comment only when the comment belongs to that user.

Likes

The like workflow is:



This allows the same user to toggle their interaction with a post between liked and unliked.

12. Forms and Validation

Django forms are used to collect and validate user input.

The **users** application contains forms for:

- Registration
- Login
- User information update
- Profile update
- Password update

UserUpdateForm

The **UserUpdateForm** is connected to Django's **User** model and manages:

- Username
- Email

ProfileUpdateForm

- The **ProfileUpdateForm** is connected to the Profile model.
- The **user** field is excluded because the Profile should remain associated with its existing User.

Date of Birth Validation

The Profile form contains custom validation for the date of birth.

The validation checks that:

- The date is not in the future.
- The calculated age does not exceed 100 years.
- The user meets the application's minimum age requirement.

Invalid values raise a Django **ValidationError**.

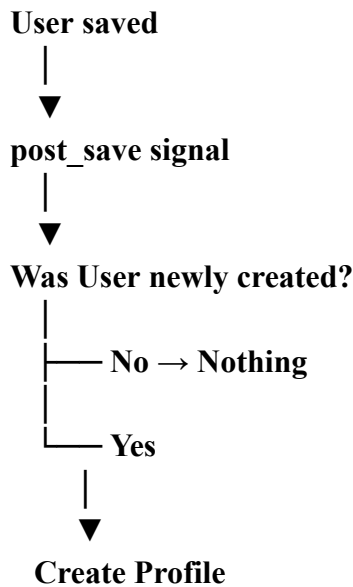
Password Update

The password update functionality uses Django's **PasswordChangeForm** to validate and process password changes.

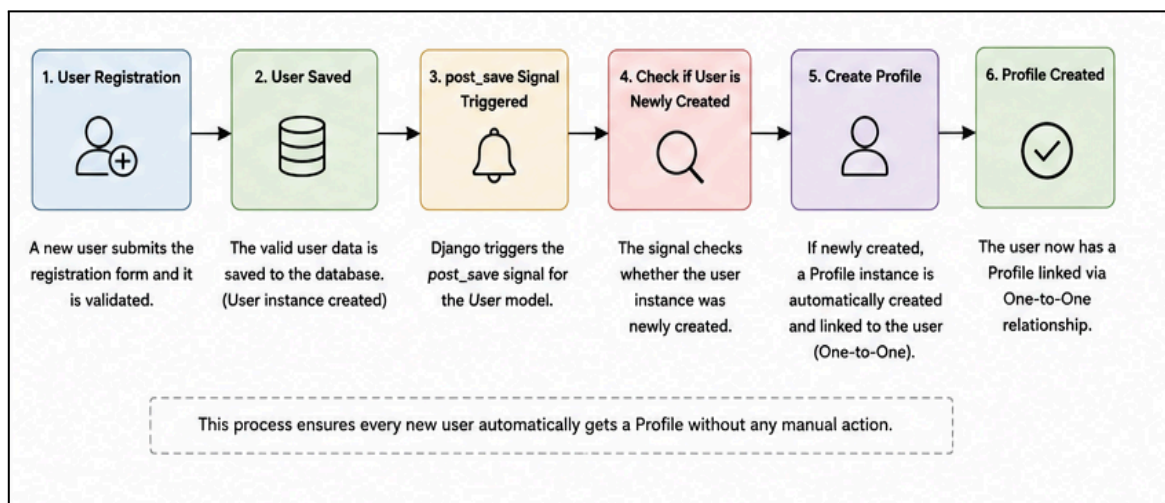
13. Django Signals

- The project uses a Django **post_save** signal for automatic Profile creation.
- The signal listens for a User being saved.

The logic is:



- This design ensures that a Profile is automatically created whenever a new User account is created.
- The signal is loaded through the **UsersConfig.ready()** method.



14. URL Routing

- The application separates URL patterns according to Django applications.
- The Users application provides routes for:

URL purpose	Function
Home	Home page
Profile	View profile
Edit Profile	Edit profile
Register	Create account
Login	Authenticate user
Logout	End session

- The Posts application provides routes for post-related operations such as:
 - Viewing posts
 - Creating posts
 - Editing posts
 - Deleting posts
 - Likes
 - Comments
 - Comment deletion
 - Named URL patterns are used by the templates when creating navigation links and submitting actions.
 - This avoids hard-coding application URLs inside templates.
-

15. Templates and Presentation Layer

- The project uses Django Templates for rendering HTML pages.
- The templates contain:
 - Navigation links
 - Forms
 - User information
 - Post information
 - Comments
 - Like controls
 - Django message display
- Django template tags such as URL reversing and template variables are used to connect the presentation layer with the application.
- For example, navigation links use named URL patterns rather than hard-coded application paths.
- The templates also use conditional rendering to display optional content.

Examples include:

- Displaying a post title only when one exists.
 - Displaying a post image only when an image exists.
 - Displaying the edited timestamp only when a post has been edited.
 - Displaying Edit/Delete controls only for content owned by the current user.
-

16. Media and Static Files

The application supports user-uploaded media.

Profile Images

- Profile pictures are stored using the Profile model's image field.

Post Images

- Post images are stored using the Post model's image field.
- The HTML forms that upload images use:

multipart/form-data

- This allows files to be transmitted together with normal form data.
 - During development, Django's media configuration is used to serve uploaded media.
 - Static files are maintained separately from uploaded media.
-

17. Messages and User Feedback

- The application uses Django's messages framework to provide feedback after user actions.
 - Messages are displayed in the templates and can indicate different types of outcomes.
 - Examples include:
 - Successful registration
 - Successful profile modification
 - No changes made
 - Invalid submitted information
 - Incorrect login credentials
 - This provides immediate feedback without requiring a separate result page for every operation.
-

18. Security and Access Control

The project uses several Django mechanisms to protect application functionality.

Authentication Protection

- Protected views use **login_required**.

- Unauthenticated users are redirected to the Login page.

CSRF Protection

- POST forms include Django's CSRF token mechanism.
- This protects form submissions against Cross-Site Request Forgery.

Ownership Validation

- Post modification and deletion are restricted to the user who owns the post.
- Comment deletion is similarly restricted to the user who owns the comment.
- The templates also hide Edit and Delete controls for posts that belong to other users.

Form Validation

- User-provided information is passed through Django forms and validation before being saved.

19. Current Implementation Status

The current v1.0 implementation includes:

Feature	Status
User registration	Implemented
User login	Implemented
User logout	Implemented
Profile creation	Implemented
Profile editing	Implemented
Profile picture	Implemented
User information editing	Implemented
Password change form	Implemented
Post creation	Implemented

Post editing	Implemented
Post deletion	Implemented
Post image upload	Implemented
Comments	Implemented
Comment deletion	Implemented
Likes	Implemented
Unlikes	Implemented
Like count	Implemented
Authentication protection	Implemented
Ownership checks	Implemented
Friends system	Not implemented
Friend-based Home feed	Not implemented
Notifications	Not implemented
WebSocket notifications	Not implemented
REST API	Not implemented
Automated tests	Not implemented

20. Known Limitations

The current implementation has several limitations.

Home Feed

- The Home page does not currently provide a personalized social feed.
- The intended friend-based feed depends on the Friends functionality, which has not yet been implemented.

Friends

- The Friends application is retained for future development but currently does not provide the friend relationship functionality.

Notifications

- The project does not currently provide notification functionality or real-time WebSocket communication.

REST API

- No REST API has been implemented in the current version.

Automated Tests

- Automated tests have not been implemented in the current version.

Development Environment

- The project currently uses SQLite and Django's development environment.
-

21. Future Enhancements

The project can be extended with the following features.

Friends and Relationships

A future Friends system can provide:

- Friend requests
- Accept/reject functionality
- Friend lists
- Relationship management
- Friend-based content feeds

Home Feed

- Once the Friends system is implemented, the Home page can be extended to display posts from the user's friends.

Notifications

A notification system can be introduced for events such as:

- Friend requests
- Likes
- Comments
- Other user interactions

Real-time notifications can eventually be implemented using WebSockets.

REST API

A REST API can provide programmatic access to application resources such as:

- Users
- Profiles
- Posts
- Comments
- Likes
- Friends

Automated Testing

Automated unit and integration tests can be added to verify:

- Models
- Forms
- Views
- Authentication
- Permissions
- Post operations

- Comment operations
- Like functionality

Frontend Improvements

The user interface can be further improved through:

- Responsive design
 - Improved navigation
 - Better interaction feedback
 - Additional accessibility improvements
-

22. Development Decisions and Lessons Learned

The project provided practical experience with several Django concepts.

Separate Profile Model

- Instead of replacing Django's built-in User model, the project uses a separate Profile model connected through a one-to-one relationship.
- This allows the built-in authentication system to remain responsible for authentication while additional user information is maintained separately.

Automatic Profile Creation

- A **post_save** signal was used to automatically create a Profile whenever a new User is created.
- This keeps Profile creation independent from the registration view.

Model-Based Forms

- Django ModelForms were used for updating User and Profile data.
- This reduces the amount of manual database-handling logic required by the views while still allowing custom validation.

Authentication-Based Access

- Django's authentication system and `login_required` decorator are used to restrict access to protected functionality.

Ownership-Based Operations

- The project applies ownership checks when users attempt to modify or delete content.
- This demonstrates the difference between simply being authenticated and being authorized to modify a particular resource.

Separation of Applications

- User-related and post-related functionality are separated into different Django applications.
- This keeps responsibilities organized and provides a structure that can be extended as additional features are introduced.

Media Handling

- The project distinguishes uploaded media from static resources and uses multipart form submissions for image uploads.

23. Conclusion

- The Django Social Media Platform v1.0 implements the core functionality of a social media web application using Django.
- The project demonstrates practical use of:
 - Django MVT architecture
 - Django ORM

- Built-in authentication
 - ModelForms
 - Custom validation
 - Signals
 - URL routing
 - Templates
 - File uploads
 - Access control
 - User-generated content
 - Database relationships
- The current implementation provides a functional foundation for user accounts, profiles, posts, comments, and likes.
 - The project is also structured to allow future expansion through the planned Friends system, notifications, REST API, automated testing, and other social networking features.
 - The current version therefore represents the completed core implementation while leaving a clear architectural path for future development.